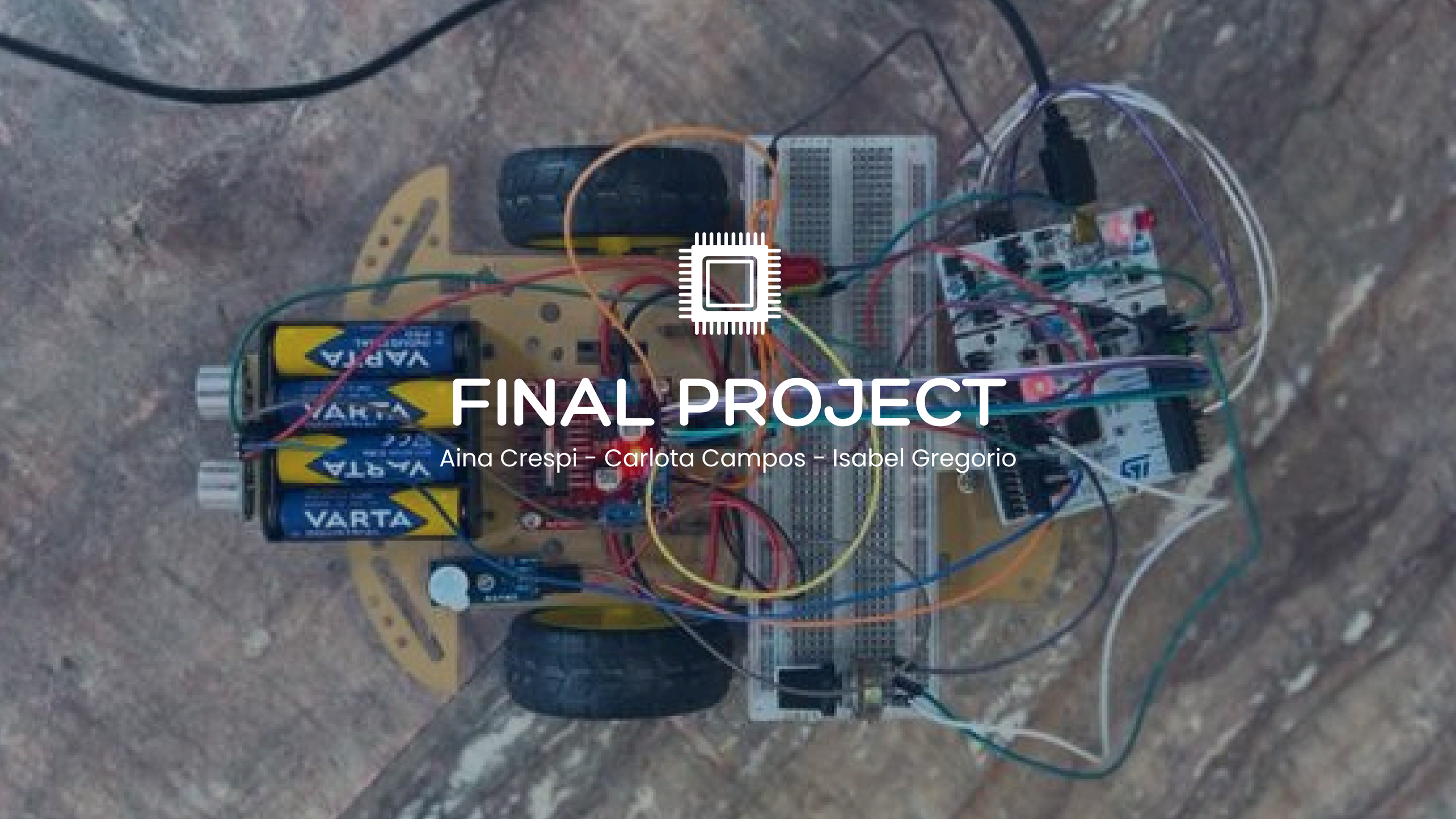
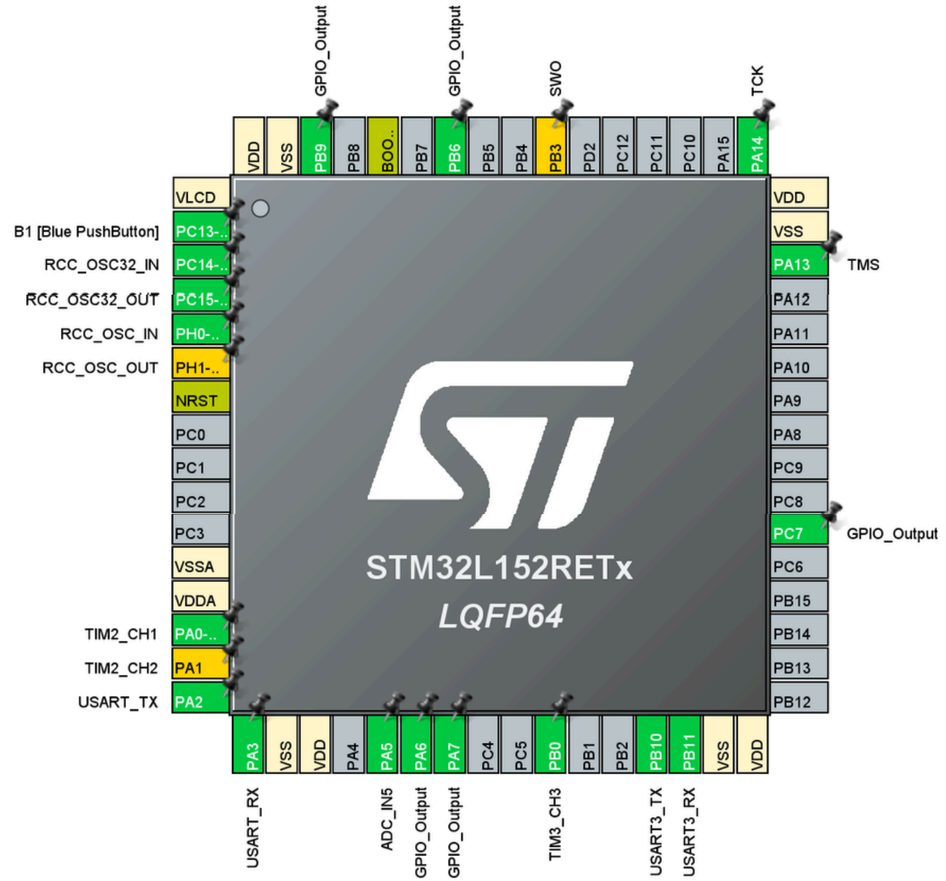
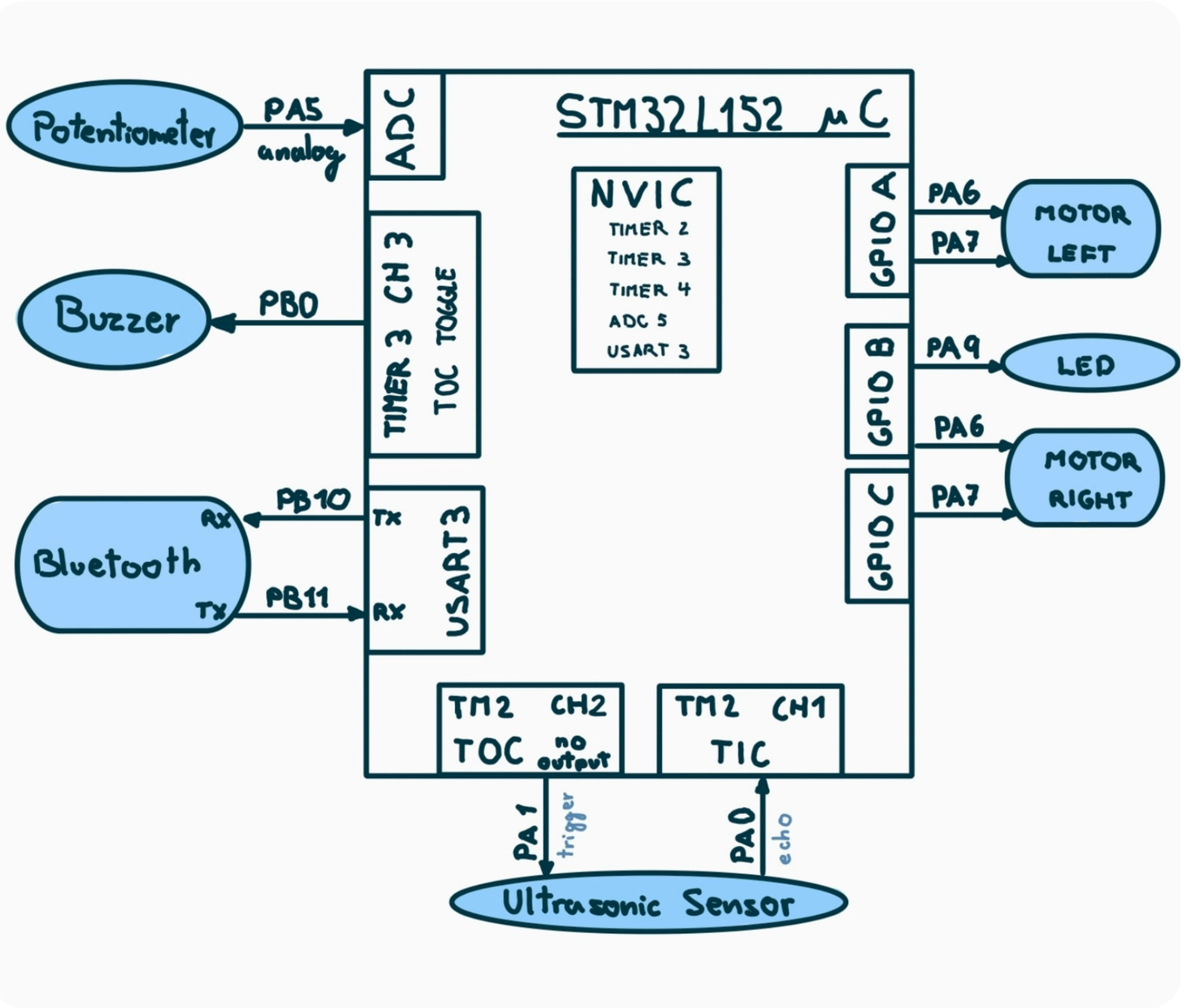


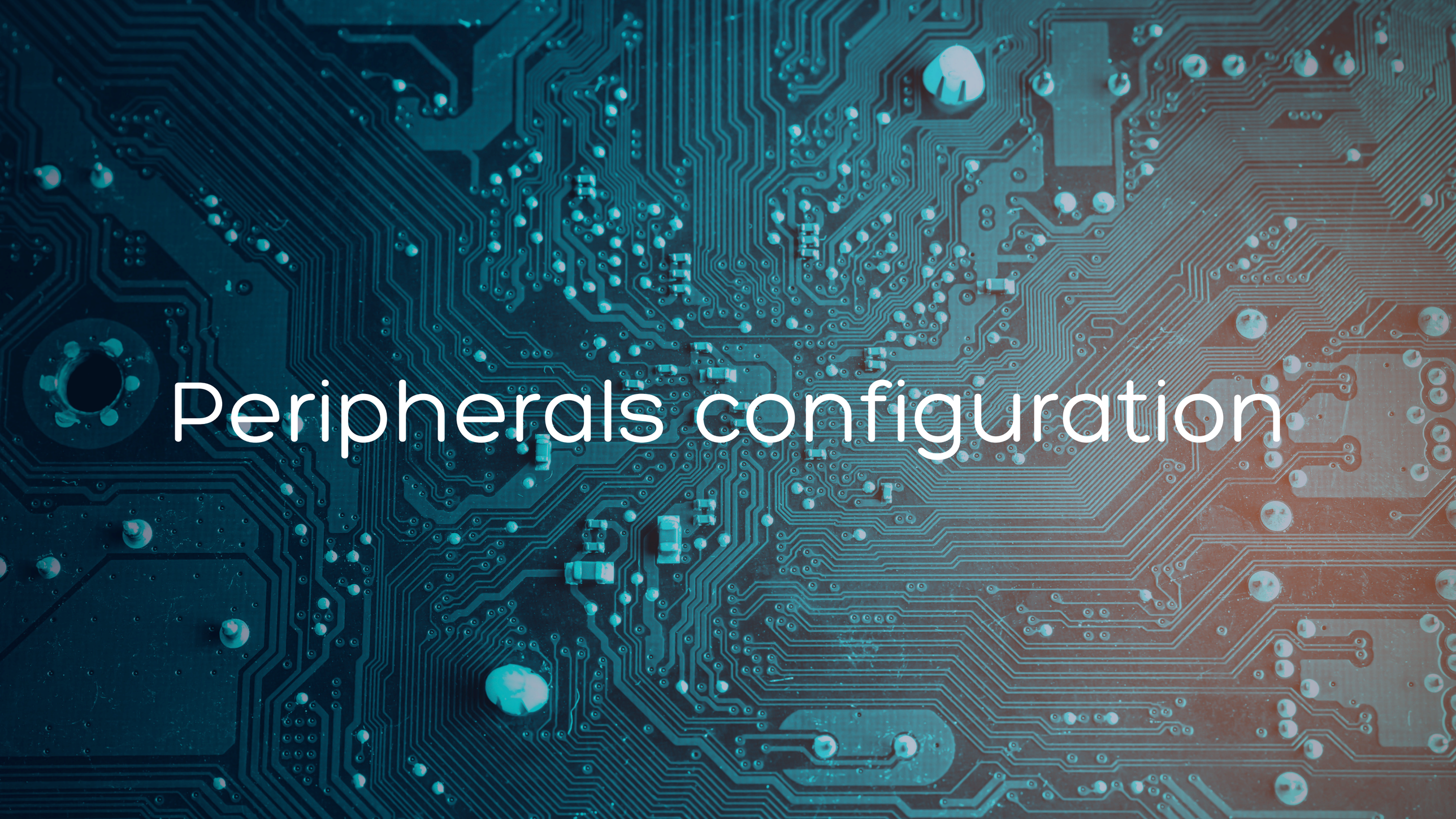
FINAL PROJECT

Aina Crespi - Carlota Campos - Isabel Gregorio

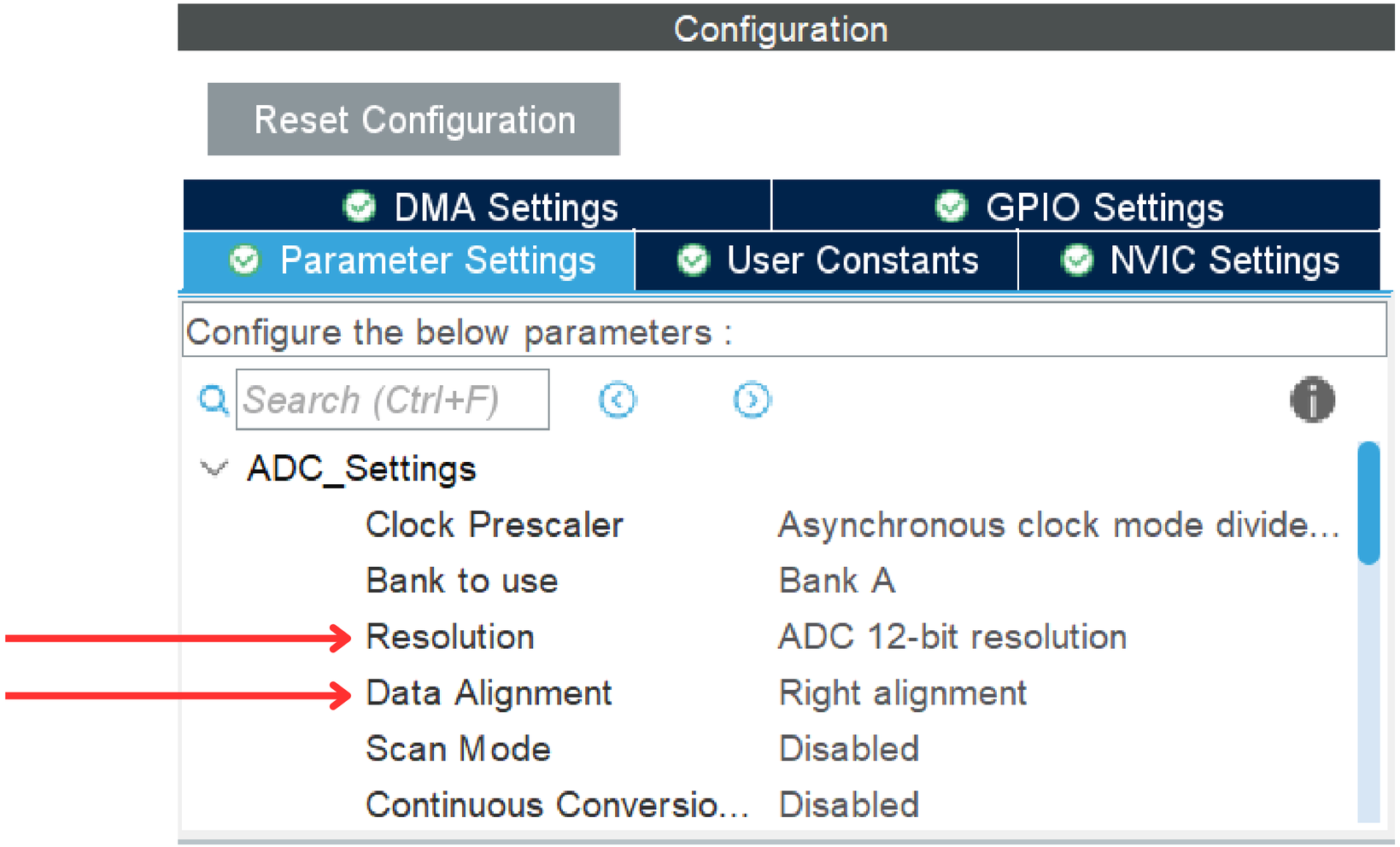
BLOCK DIAGRAM



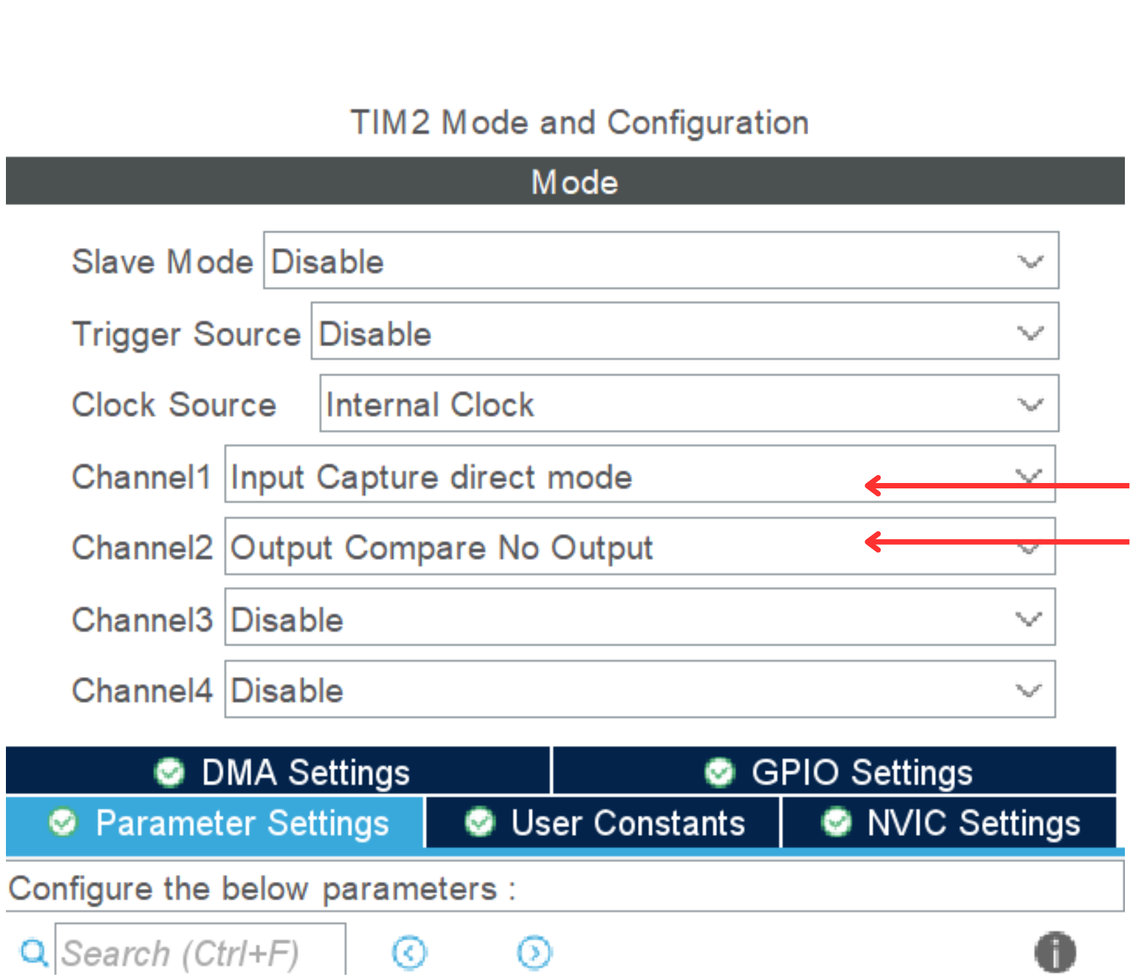


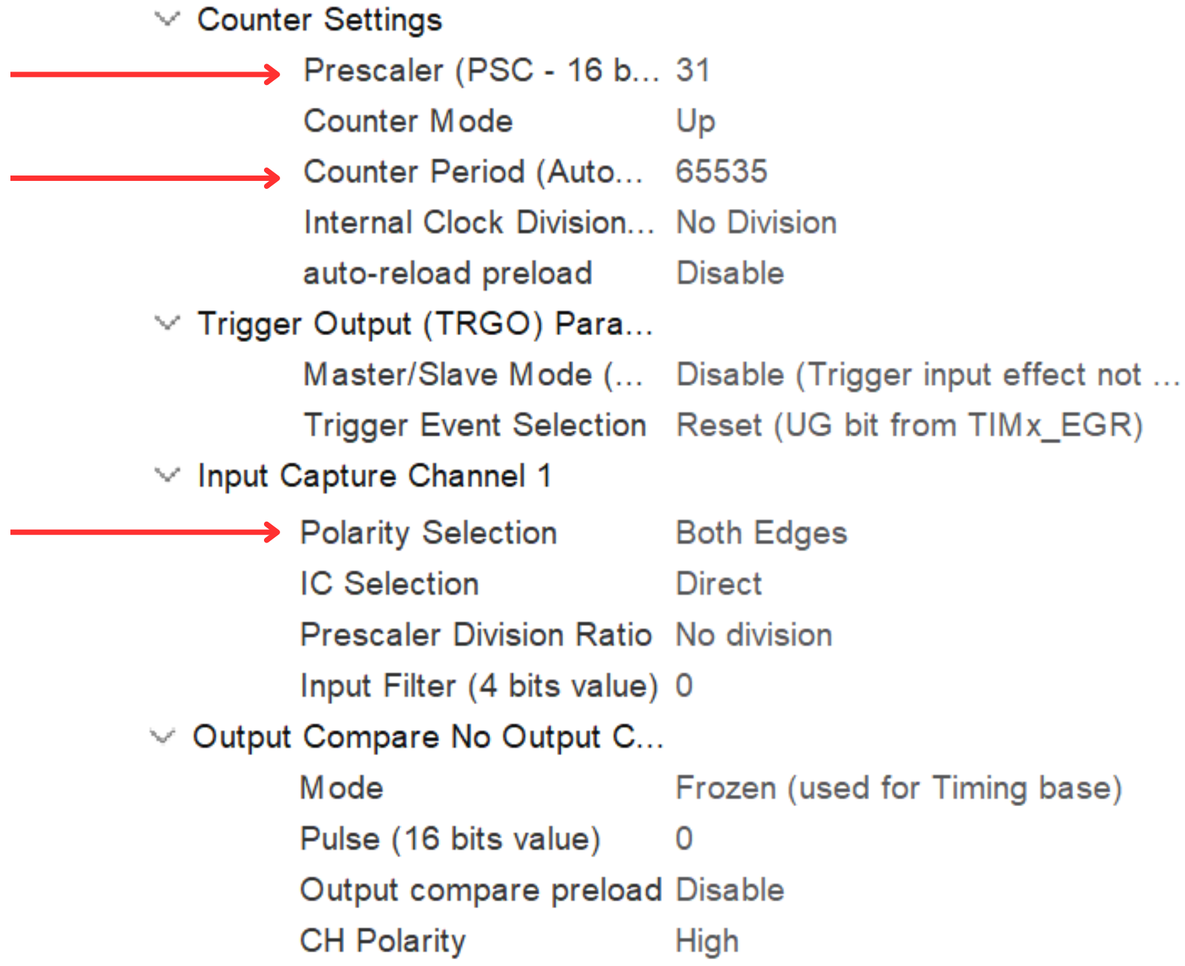

Peripherals configuration

ADC IN5

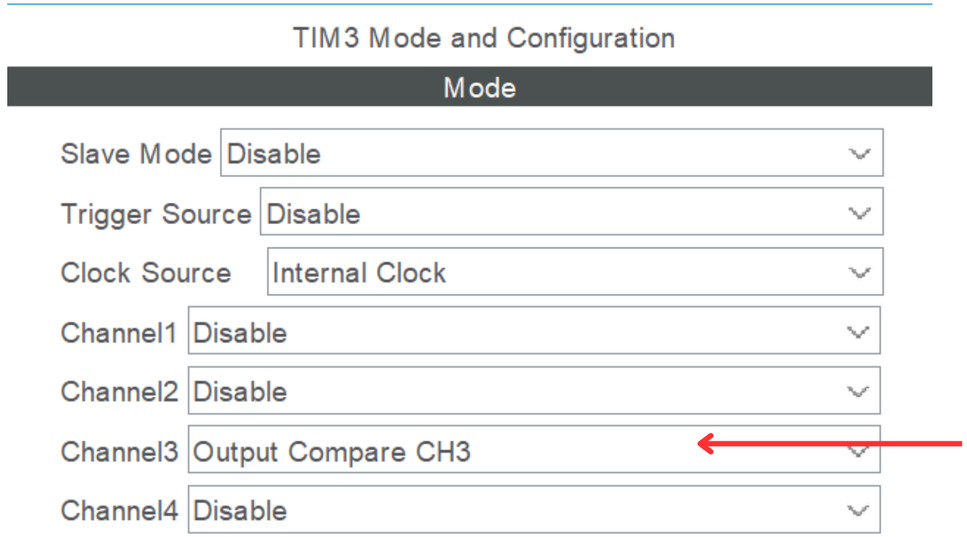

Configuration
Reset Configuration
DMA Settings
GPIO Settings
Parameter Settings
User Constants
NVIC Settings
Configure the below parameters :
Search (Ctrl+F)
ADC_Settings
Clock Prescaler
Asynchronous clock mode divide...
Bank to use
Bank A
Resolution
ADC 12-bit resolution
Data Alignment
Right alignment
Scan Mode
Disabled
Continuous Conversio...
Disabled

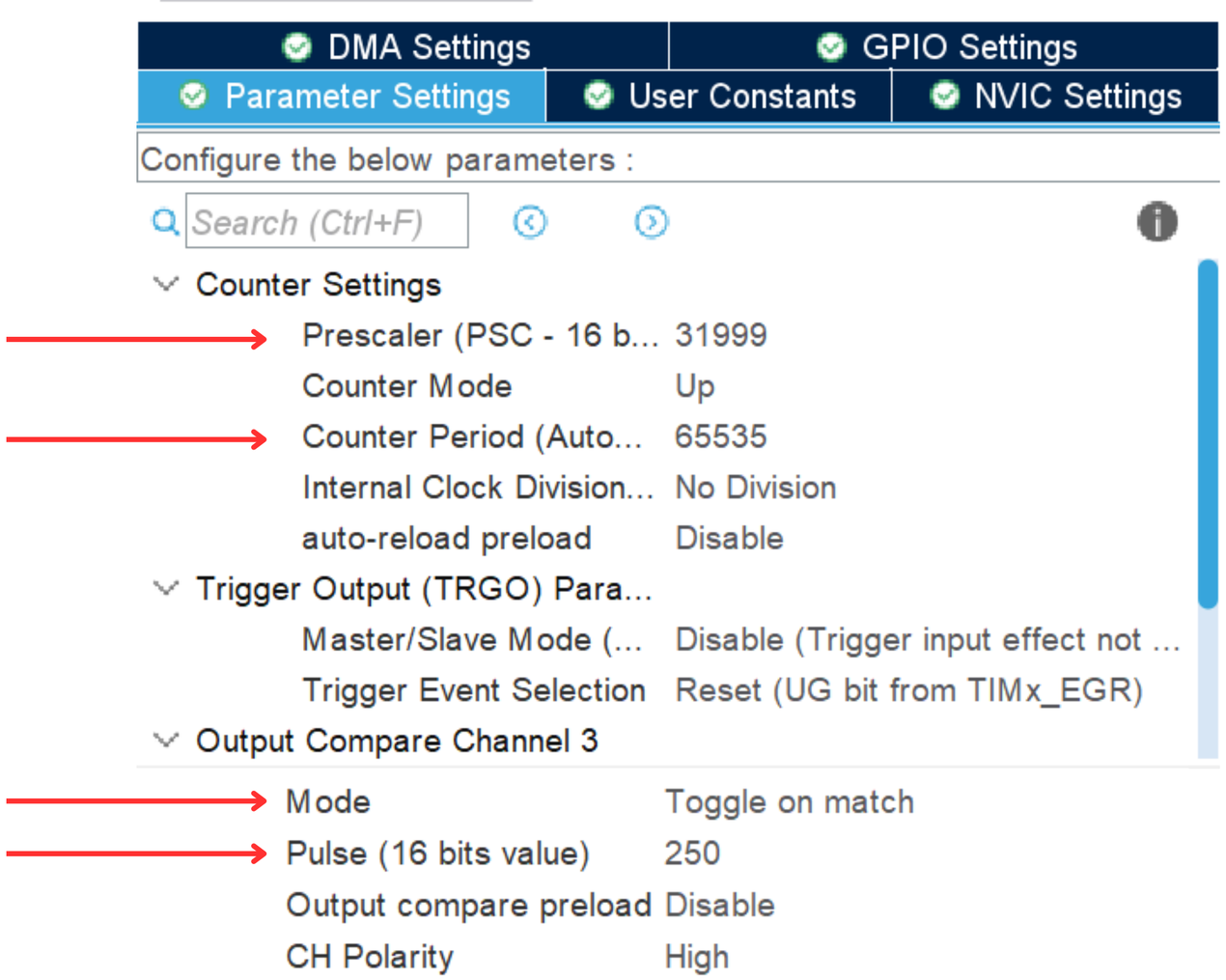
TIM 2: ultrasonic sensor



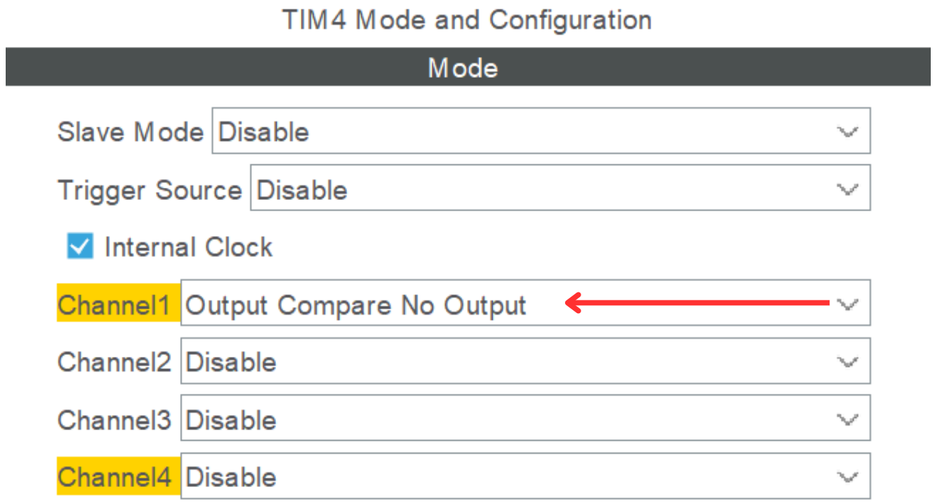


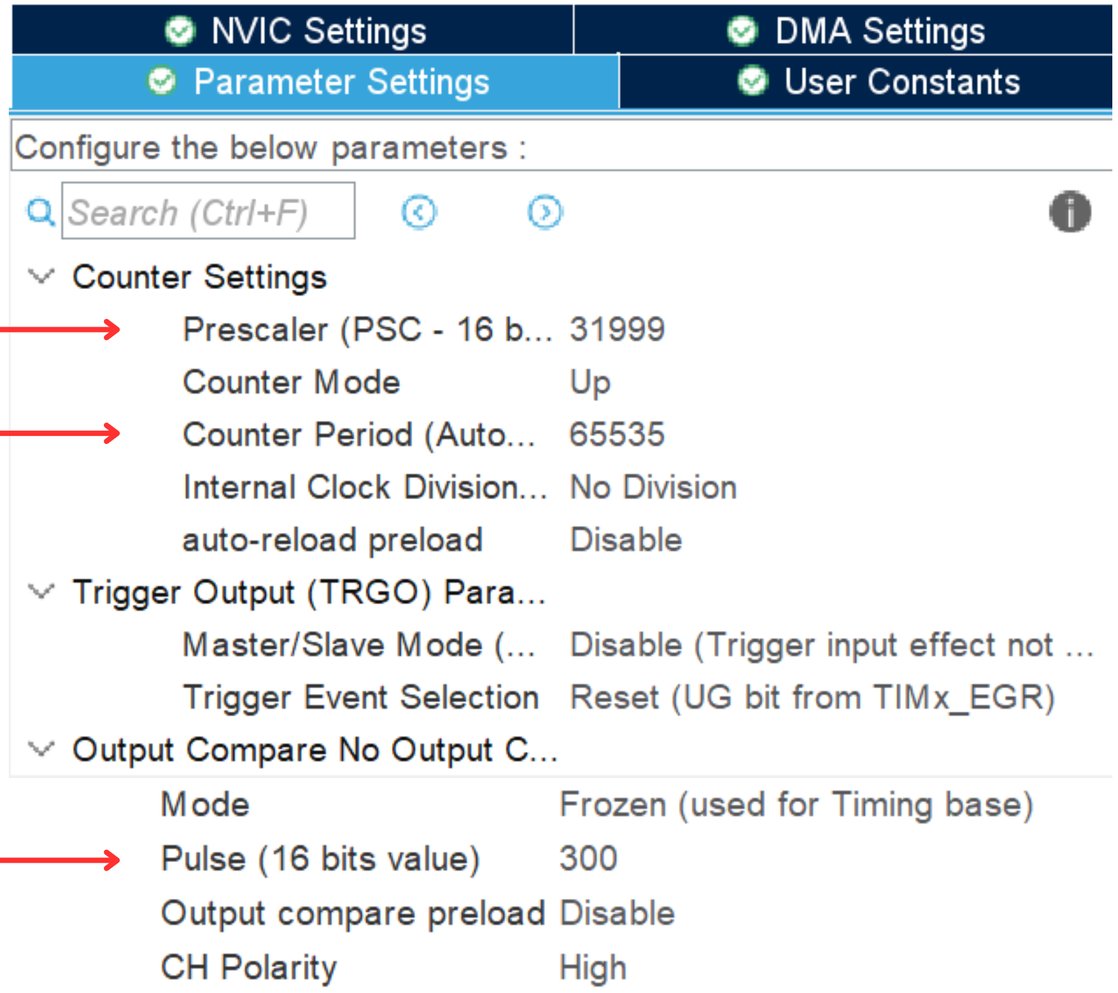
TIM 3: toggle for buzzer



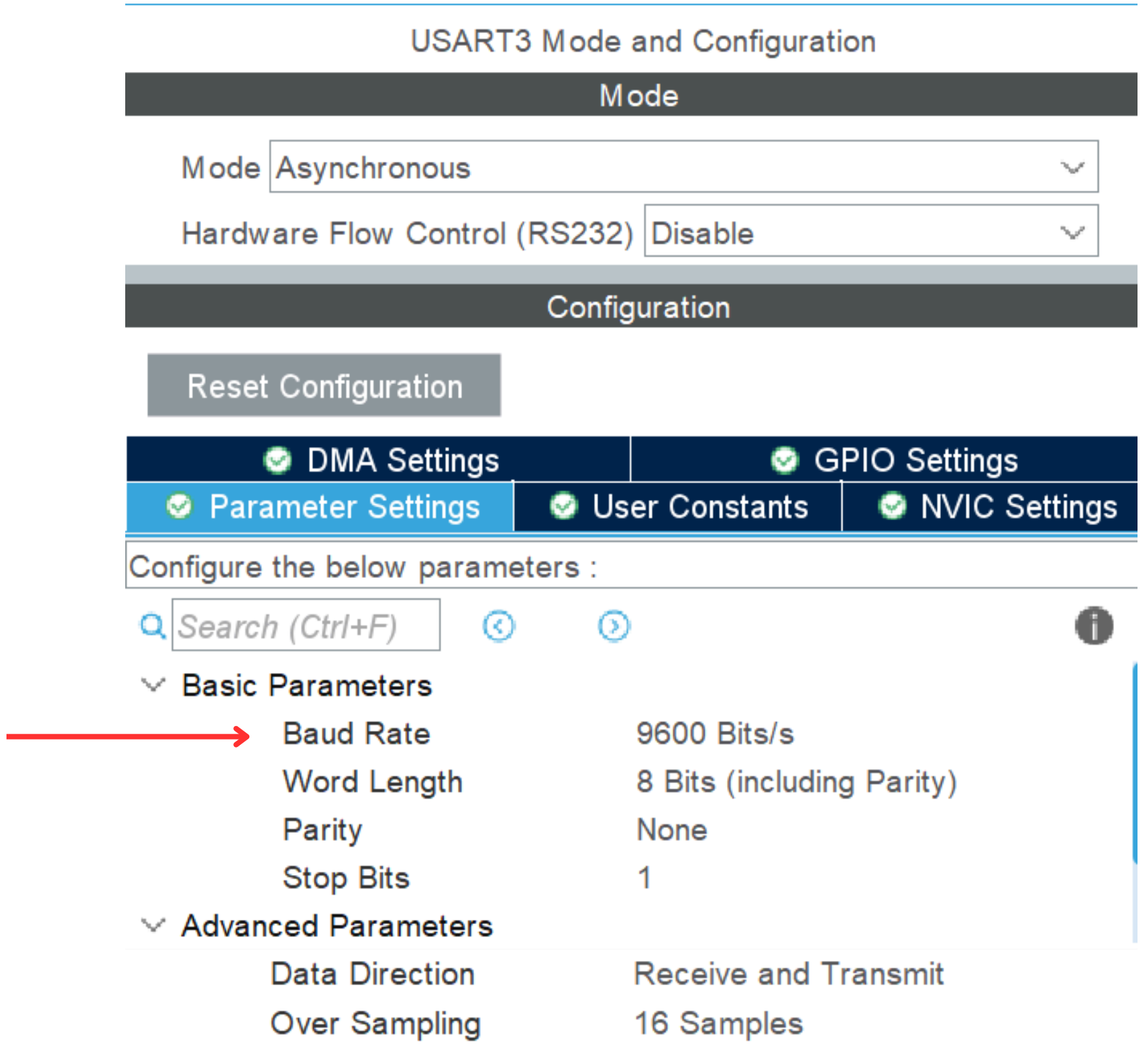


TIM 4: for mesure every 300 ms

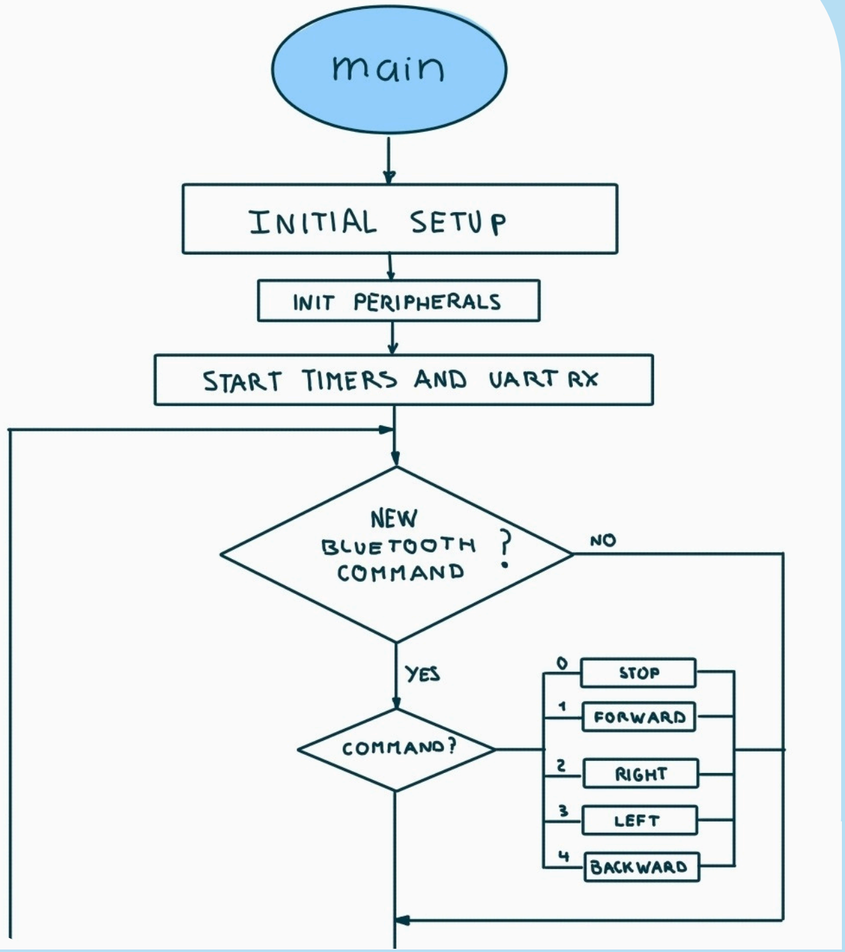


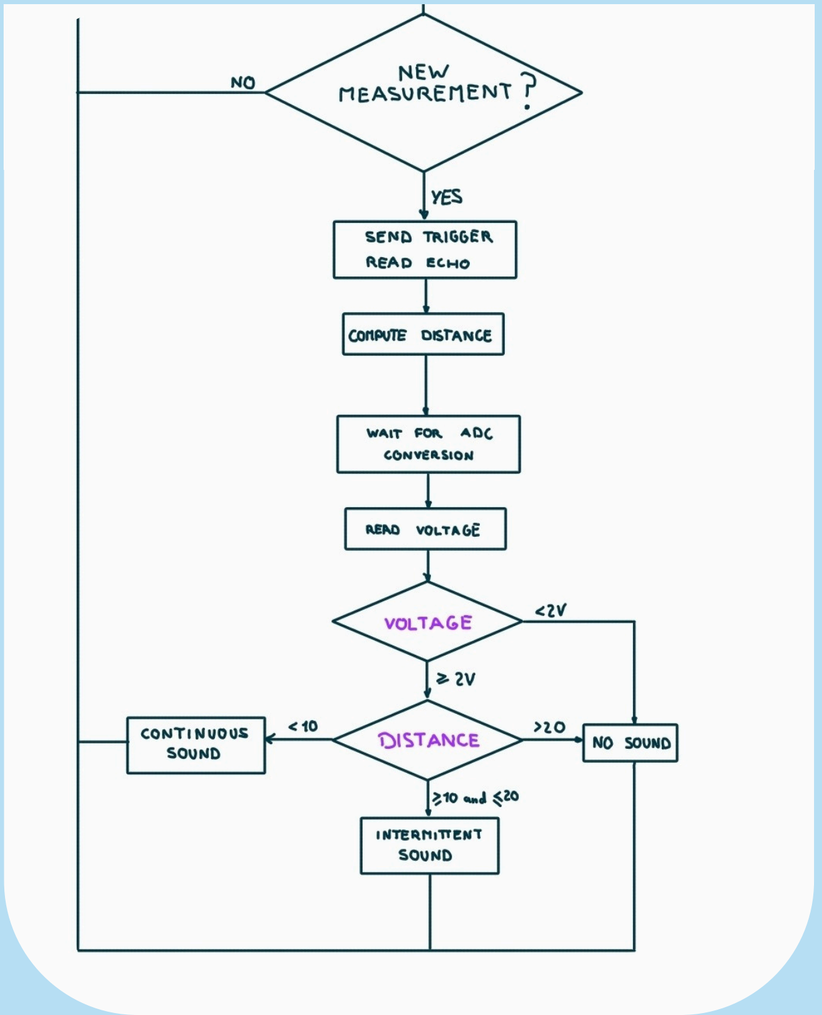


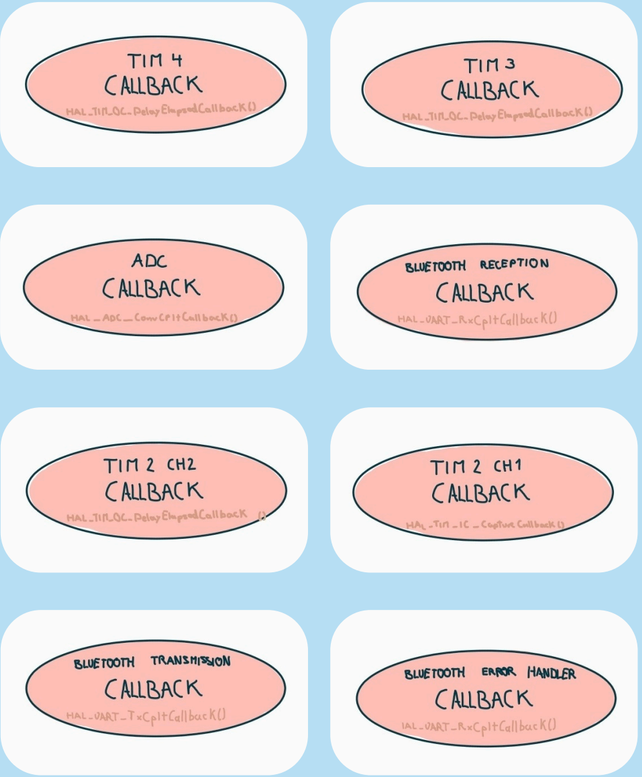
USART 3

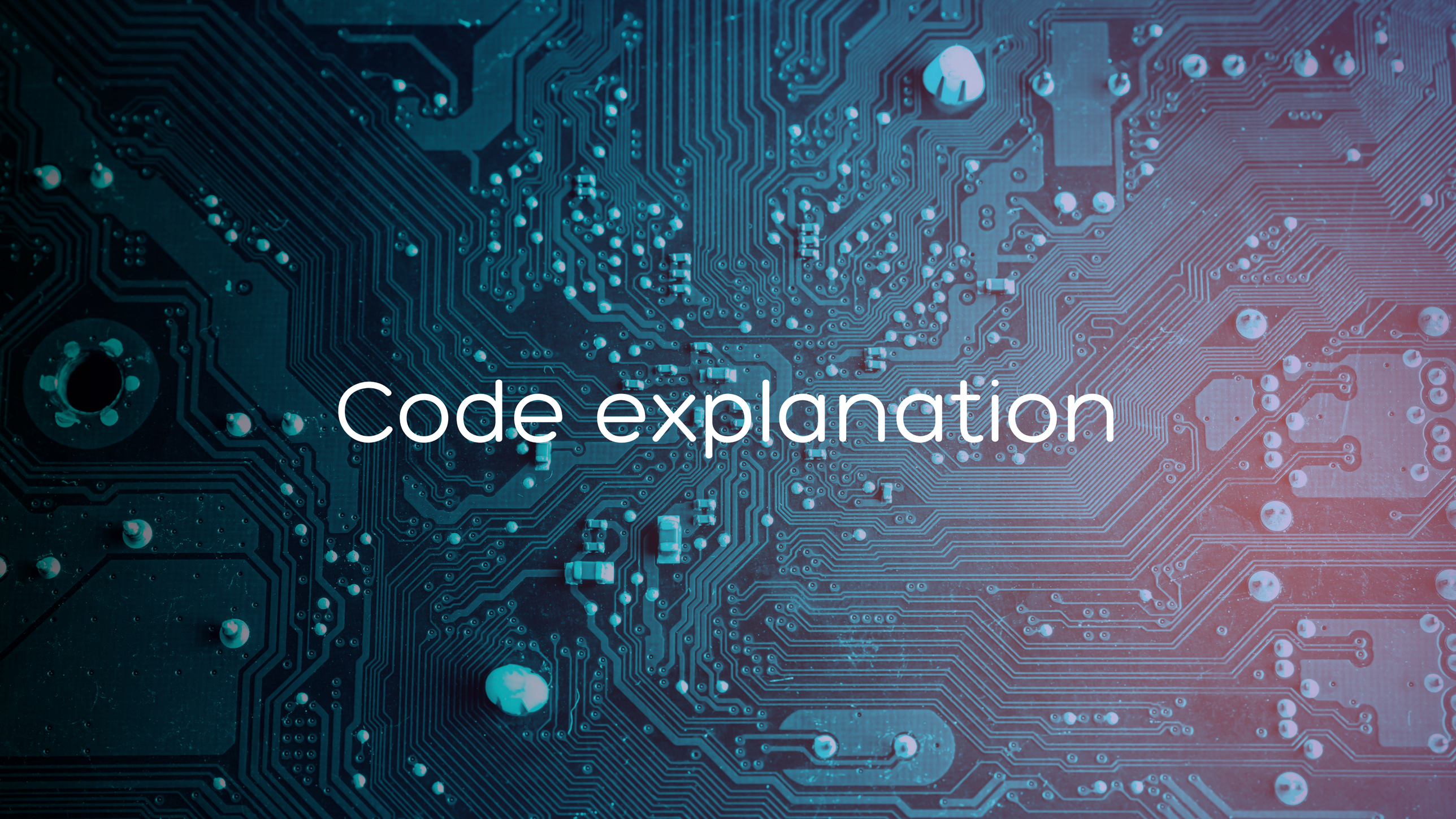

USART3 Mode and Configuration
Mode
Mode
Asynchronous
Hardware Flow Control (RS232)
Disable
Configuration
Reset Configuration
DMA Settings
GPIO Settings
Parameter Settings
User Constants
NVIC Settings
Configure the below parameters :
Search (Ctrl+F)
Basic Parameters
Baud Rate
9600 Bits/s
Word Length
8 Bits (including Parity)
Parity
None
Stop Bits
1
Advanced Parameters
Data Direction
Receive and Transmit
Over Sampling
16 Samples

Flowchart


main
INITIAL SETUP
INIT PERIPHERALS
START TIMERS AND UART RX
NEW BLUETOOTH COMMAND ?
NO
YES
COMMAND?
0 STOP
1 FORWARD
2 RIGHT
3 LEFT
4 BACKWARD


NO
NEW MEASUREMENT ?
YES
SEND TRIGGER READ ECHO
COMPUTE DISTANCE
WAIT FOR ADC CONVERSION
READ VOLTAGE
VOLTAGE
<2V
≥ 2V
CONTINUOUS SOUND
<10
DISTANCE
>20
NO SOUND
≥10 and ≤20
INTERMITTENT SOUND


TIM 4 CALLBACK
HAL_TIM_OC_DelayElapsedCallback()
TIM 3 CALLBACK
HAL_TIM_OC_DelayElapsedCallback()
ADC CALLBACK
HAL_ADC_ConvCpltCallback()
BLUETOOTH RECEPTION CALLBACK
HAL_UART_RxCpltCallback()
TIM 2 CH2 CALLBACK
HAL_TIM_OC_DelayElapsedCallback()
TIM 2 CH1 CALLBACK
HAL_TIM_IC_CaptureCallback()
BLUETOOTH TRANSMISSION CALLBACK
HAL_UART_TxCpltCallback()
BLUETOOTH ERROR HANDLER CALLBACK
HAL_UART_RxCpltCallback()


Code explanation

TIM callback

```c
void HAL_TIM_OC_DelayElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM4){
        TIM4->CCR1 = TIM4->CNT + 300; //timer for measuring every 300ms
        measure = 1;

        HAL_ADC_Start_IT(&hadc); //start the adc conversion
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_SET); // tigger

        __HAL_TIM_SET_COUNTER(&htim2, 0);
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, 12);

        //start the Output Compare interrupt for Channel 2
        HAL_TIM_OC_Start_IT(&htim2, TIM_CHANNEL_2);

    }
    if (htim->Instance == TIM3){
        TIM3->CCR3 = TIM3->CNT + 250; //buzzer toggles every 0.25 s
    }
    if(htim->Instance == TIM2 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2) {
        //change pin a1 to 0
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_RESET);

        //Stop the interrupt so it doesn't repeat
        HAL_TIM_OC_Stop_IT(&htim2, TIM_CHANNEL_2);
    }
}
```

Mesure 300 ms to start mesuring, start ADC conversion, start trigger

Toggle of buzzer

Stop the trigger

ADC callback

```c
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    adc_new_value = HAL_ADC_GetValue(hadc);
    sample_ready = 1;
    HAL_ADC_Stop_IT(hadc);
}
```

When the ADC finishes an interrupt conversion, we save the value in adc_new_value, notify sample_ready and stop the ADC

TIM callback

```c
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim){
   if(htim->Instance==TIM2 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1) {
       i++;
       if (i==1){
               t1=__HAL_TIM_GET_COMPARE (&htim2, TIM_CHANNEL_1); //read CCR1
               //t1=HAL_TIM_ReadCapturedValue (&htim4, TIM_CHANNEL_1);
       }else{
               i=0;
               t2=__HAL_TIM_GET_COMPARE (&htim2, TIM_CHANNEL_1);
               if(t1>t2){
                   time=65536-t1+t2; //overflow
               }else{
                   time=t2-t1; //no overflow
               }
               got_time = 1;
       }
   }
}
```

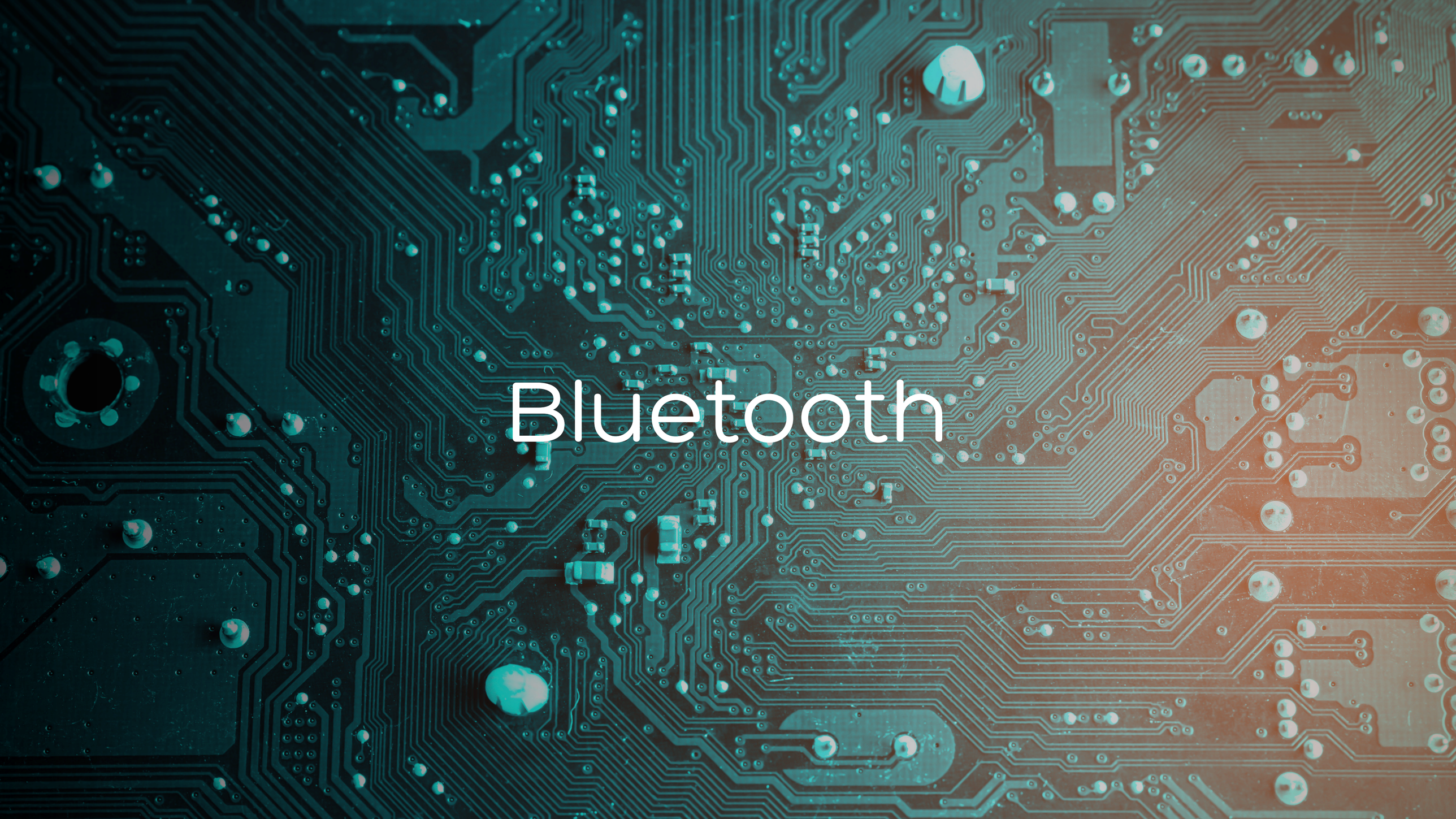
Callback to measure echo time

main

```
if (measure == 1){
        measure = 0;

        //wait for the echo channel to measure time
        while(got_time==0){
        }
        got_time = 0;
        distance = ((float)time)/58.0;

        // wait for the analog input to be converted
        while(sample_ready == 0){
        }
        sample_ready = 0;
        voltage = 3.0f * adc_new_value/ 4095.0f;

        if(voltage >= 2.0f){
            if(distance > 20.0f){
                //no sound
                TIM3->CCMR2 &= ~(0b111 << 4);
                TIM3->CCMR2 |=  (0b100 << 4); //force inactive mode to the buzzer

            }
            else if(distance >= 10.0f && distance <= 20.0f){
                //intermittent buzzer
                TIM3->CCMR2 &= ~(0b111 << 4);
                TIM3->CCMR2 |=  (0b011 << 4); //return to toggle mode

            }
            else{
                //continuous sound
```

Once we have the measurements, we execute the corresponding case:

- No sound
- Intermittent sound
- Continuous sound


Bluetooth

USART callback

```c
// Callback function when data is received
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART3) {
            bt_last = bluetooth_rx_buffer[0];   // snapshot

            HAL_UART_Receive_IT(&huart3, bluetooth_rx_buffer, 1); // restart

            bluetooth_data_ready = 1;
    }
}
```

Receives the bit

Sets “bluetooth_data_ready” to 1

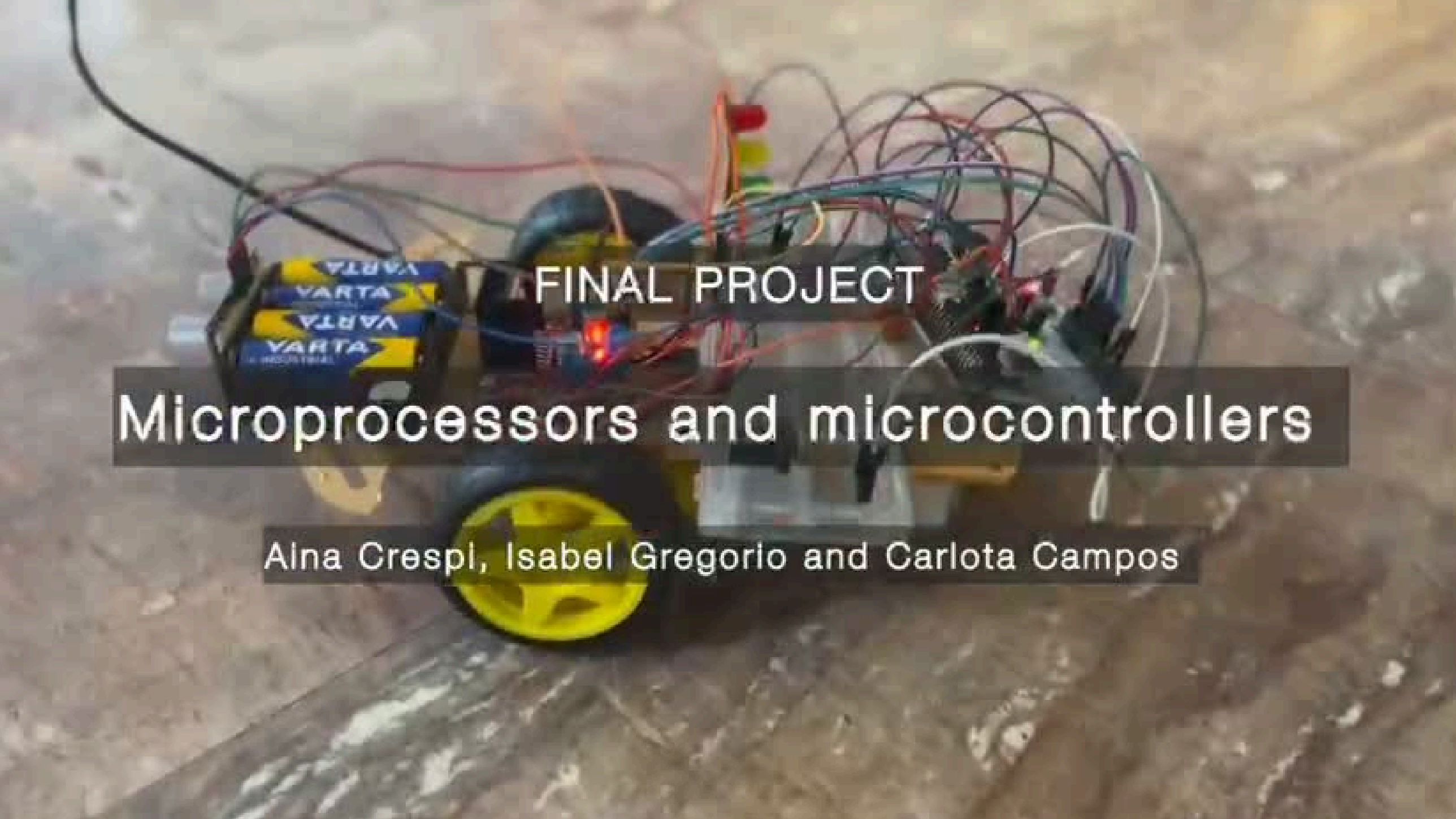
And restarts

main

Once we have the message, we execute the corresponding case:

- 0 -Stop
- 1 -Forward
- 4 -Backward
- 2 -Left
- 3 -Right

```
if (bluetooth_data_ready == 1) {
    bluetooth_data_ready = 0;

    uint8_t c = bt_last;

    if (c == '\r' || c == '\n'){

    }else{

        HAL_UART_Transmit(&huart3, &c, 1, 100);

    // Process command from Bluetooth
    switch(c) {
        case '1':
            // Forward

            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_9, GPIO_PIN_SET);

            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);

            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_7, GPIO_PIN_RESET);
            break;
        case '0':
            // Stop
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_9, GPIO_PIN_RESET);

            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
```


FINAL PROJECT
Microprocessors and microcontrollers
Alna Crespi, Isabel Gregorio and Carlota Campos